Product Backlog

6주차 · 제품 요구사항 목록 (우선순위 정렬)

프로젝트명	NISQ 환경에서 QAOA를 이용한 Max-Cut 근사 탐색과 오류 완화 분석

조	3 조
지도교수	남병규 교수 (서명)
조원	202302611 정민주 / 202100986 최정호 / 202303514 최지원
제출일자	2026. 10. 18.
GitHub 저장소	https://github.com/minju43/Team3

Document Revision History

Rev#	Date	Affected Section	Author
1	2026/10/15	최초 작성	정민주

0. Backlog Document Header

Project Name	NISQ 환경에서 QAOA를 이용한 Max-Cut 근사 탐색과 오류 완화 분석
Version	v1.0
작성일 / 수정일	2026.10.15 / 2026.10.16
Product Owner (PO)	정민주
Scrum Master (SM)	최지원
Developers (팀원)	최정호

1. Product Vision & Success Metrics

1.1 Product Vision

양자 알고리즘을 학습 및 연구하는 학생, 연구자, 최적화 문제의 양자 하드웨어 적용 가능성을 검토하는 소프트웨어 엔지니어가 주요 사용자이다. 소셜 네트워크 친밀도 기반 Max-Cut이 지닌 개념적 왜곡을 바로잡고 NISQ 장비의 노이즈로 인해 QAOA 최적화 해가 붕괴되는 현상을 이론과 실제 장비 사이에서 직관적으로 검증하기 어렵다는 문제를 해결한다. 실제 IBM Quantum QPU 환경에서 노이즈에 의해 왜곡된 해와 오류 완화 기술로 복원된 해를 스위치 하나로 실시간 비교 및 시각화할 수 있으며 고전 알고리즘 대비 양자 연산의 복원율과 근사비를 한눈에 입증하는 4자 정량 벤치마크 리포트를 제공하여 양자 컴퓨팅의 현실적인 유효성을 직관적으로 전달한다.

1.2 Success Metrics

번호	지표	목표치	측정 방법 · 시점

번호	지표	목표치	측정 방법 · 시점
1	QAOA 근사비 달성률	파티 갈등 회피 그래프 (N = 8~14) 대상 근사비 ≥ 0.70 달성	Brute-force 이론적 최적해 대비 상태벡터 시뮬레이터 산출 기댓값 비율 계산 알고리즘 시뮬레이션 및 검증 단계(2027년 1월)
2	실제 QPU 오류 완화 성능 복원율	Raw QPU 대비 오류 완화 적용 시 근사비 또는 정답 비트열 확률 ≥ 20% 향상	동일한 IBM Quantum 하드웨어 백엔드에서 Raw 실행값과 보정 실행값의 기댓값 비교 측정 QPU 벤치마크 및 오류 완화 검증 단계(2027년 3월)
3	인터랙티브 웹 플랫폼 시각화 응답시간	노드 10개 기준 파티 분할 시각화 및 프리셋 로딩 1.5초 이내	웹 대시보드 갈등 관계 빌더에서 2분할 렌더링 시 브라우저 응답 시간 10회 측정 평균 Sprint 2 UI/UX 시연 단계
4	4자 벤치마크 파이프라인 정합성 통과율	사전 정의 시나리오 대상 테스트 통과율 100%	고전 알고리즘 2개와 양자 알고리즘 2개 간 4자 비교 데이터 수집 자동화 테스트 실행 통합 파이프라인 검증 및 최종 시연 시점(2027년 5월)

2. 전체 Epic 구조 (도메인 기준)

Epic ID	Epic 이름	설명	양식 4 §2.3의 기능 (코어/부가)
E1	파티 갈등 문제의 수학 공식 변환	파티 참석자 간의 갈등 관계 데이터를 양자 컴퓨터가 계산할 수 있는 표준 최적화 수학식으로 자동 변환하는 기능	코어

Epic ID	Epic 이름	설명	양식 4 §2.3의 기능 (코어/부가)
E2	QAOA 알고리즘 구축	갈등을 최소화하는 최적의 파티 분할 조합을 찾아내기 위한 양자-고전 협업 기본 알고리즘 파이프라인 개발	코어
E3	실제 IBM 양자 컴퓨터 하드웨어 연동	클라우드를 통해 실제 IBM 양자 컴퓨터 칩에 접속하여 파티 분할 연산을 직접 실행하고 측정값을 받아오는 기능	코어
E4	양자 하드웨어 오류 완화 엔진	실제 양자 기계의 노이즈로 인해 망가진 오답 결과를 필터링하여 정상적인 파티 배정 결과로 복원하는 보정 기능	코어
E5	비교용 일반 컴퓨터 알고리즘 개발	양자 컴퓨터 결과의 우수성을 객관적으로 검증하기 위해 일반 컴퓨터로 100% 정답 및 기존 최고 알고리즘 답을 계산해 두는 기능	부가
E6	갈등 관계 드로잉	웹 브라우저서 사용자가 마우스로 파티 참석자들을 배치하고 서로 간의 갈등 수치를 직접 선으로 연결하는 입력 캔버스 화면	부가
E7	파티 2분할 시각화	양자 연산 결과에 따라 참석자들을 A 파티와 B 파티로 분리 배치하고 회피된 갈등을 점선으로 보여 주는 화면	부가
E8	오류 복원 비교 및 성능 분석 대시보드	보정 스위치 ON/OFF에 따른 결과 복원 과정을 시연하고 일반 컴퓨터 대비 양자 컴퓨터의 성능을 그래프 차트로 비교 분석하는 화면	부가

3. 개별 Epic 내용

3.1 E1 – 파티 갈등 문제의 수학 공식 변환

US-01 갈등 그래프 데이터의 QUBO/Ising 수신 자동 변환

Epic	E1 파티 갈등 문제의 수학 공식 변환	US ID	US-01
User Story (As a [Actor] / I want [행동] / So that [가치])			
As a 양자 최적화 연구자로서 I want 웹 화면에 입력된 파티 참석자 간 갈등 관계 데이터를 양자 연산용 수학 공식으로 자동 변환할 수 있다. So that 공식을 QAOA 양자 회로에 즉각 주입할 수 있는 표준 수식 행렬을 얻을 수 있다.			
Priority	☑ Must ☐ Should ☐ Could ☐ Won't	**Estimate (SP)**	3 SP

Acceptance Criteria (Given / When / Then — 핵심 3~5개만 요약)

AC ID	Given (상황)	When (행동)	Then (결과)
AC-01	캔버스에서 참석자 수와 갈등 가중치가 포함된 인접 행렬이 전달	QUBO 변환 함수 호출	각 갈등 에지가 파티 분리 변수 형태로 매핑된 QUBO 행렬 객체가 정상 생성된다
AC-02	생성된 QUBO 객체가 준비	Qiskit의 to_ising() 변환 메서드 실행	양자 게이트 회로에 직접 적용 가능한 파울리 Z 연산자 기반 비용 해밀토니안이 도출된다
AC-03	유효하지 않은 그래프 전달	수학 공식 변환 시도	오류 예외를 반환, 사용자에게 오류 안내 메시지를 띄운다
AC-04	양자 수식 변환 완료	수식 검증 모듈 실행	Brute-force 평가 모듈과 동일한 갈등 회피 목적함수값을 산출하는 수학적 정합성이 100% 검증된다.

3.2 E2 – QAOA 알고리즘 구축

US-02 고전-양자 하이브리드 QAOA 최적화 루프 파이프라인 구현

Epic	E2 QAOA 알고리즘 구축	US ID	US-02
User Story (As a [Actor] / I want [행동] / So that [가치])			
As a 양자 최적화 연구자로서 I want Qiskit 기반으로 파라미터화된 QAOA 양자 회로를 생성하고 고전 최적화 알고리즘과 연동하여 파라미터를 자동 갱신할 수 있다. So that 파티 갈등을 최소화하는 최적의 양자 상태로 회로 매개변수를 수렴시킬 수 있다.			
Priority	☑ Must ☐ Should ☐ Could ☐ Won't	**Estimate (SP)**	5 SP

Acceptance Criteria (Given / When / Then — 핵심 3~5개만 요약)

AC ID	Given (상황)	When (행동)	Then (결과)
AC-01	E1에서 변환된 비용 해밀토니안과 지정된 회로 깊이 제공	QAOA 회로 생성 함수 실행	모든 큐비트에 중첩 상태 생성 게이트와 비용/믹서 해밀토니안 게이트가 결합된 파라미터 회로 정상 빌드된다
AC-02	초기 파라미터와 무잡음 Aer 시뮬레이터 준비 완료	고전 옵티마이저 최적화 루프 가동	목적함수가 최대화되는 방향으로 매개변수가 자동 갱신, 설정된 반복 횟수 이내에 수렴한다
AC-03	최적 파라미터 수렴 완료	최적 회로를 1024회 이상 샘플링 측정	갈등 회피 점수가 높은 상위 최적 컷 비트열들이 히스토그램 최상위 확률 분포로 검출된다
AC-04	파티 갈등 그래프에 대해 최적화 결과 도출	Brute-force 최적해 대비 근사비 산출	무잡음 시뮬레이션 환경에서 근사비 ≥ 0.70 목표치를 달성하는지 검증된다

3.3 E3 – 실제 IBM 양자 컴퓨터 하드웨어 연동

US-03 실제 IBM QPU 백엔드 작업 제출 및 원시 측정 데이터 수집

Epic	E3 실제 IBM 양자 컴퓨터 하드웨어 연동	US ID	US-03
User Story (As a [Actor] / I want [행동] / So that [가치])			
As a 양자 최적화 연구자로서 I want 클라우드를 통해 실제 IBM Quantum QPU 백엔드에 접속하여 QAOA 회로를 하드웨어 맞춤 변환하고 작업을 제출할 수 있다. So that 가상 시뮬레이터가 아닌 실제 물리적 양자 칩에서 노이즈가 반영된 원시 파티 배정 측정 데이터를 획득할 수 있다.			
Priority	☑ Must ☐ Should ☐ Could ☐ Won't	**Estimate (SP)**	5 SP

Acceptance Criteria (Given / When / Then — 핵심 3~5개만 요약)

AC ID	Given (상황)	When (행동)	Then (결과)
AC-01	유효한 IBM Quantum API 토큰이 시스템 환경 변수에 설정	IBMQ 공급자 인증 및 백엔드 목록 조회 함수 실행	실행 가능한 실제 QPU 장비 목록과 대기열 상태 정보가 정상 반환된다
AC-02	E2에서 생성된 논리적 QAOA 제공	대상 QPU 칩의 물리적 큐비트 연결 구조에 맞춰 transpile() 수행	하드웨어 네이티브 게이트 세트로 변환된 실행 가능 양자 회로 객체가 빌드된다
AC-03	트랜스파일된 회로와 샷 수 설정 완료	Qiskit Runtime Sampler 프리미티브를 통해 실제 QPU로 작업 제출	고유 Job ID가 발급되고 비동기 폴링을 통해 작업 완료 시까지 상태가 추적된다
AC-04	실제 하드웨어 작업 실행 정상 완료	원시 측정 결과 데이터를 수신	비트 판독 노이즈가 포함된 1024회의 비트열 측정 카운트 딕셔너리가 손실 없이 로컬에 캐싱 및 저장된다

3.4 E4 – 양자 하드웨어 오류 완화 엔진

US-04 실제 QPU 측정 결과에 대한 M3 및 ZNE 오류 완화 보정 처리

Epic	E4 양자 하드웨어 오류 완화 엔진	US ID	US-04
User Story (As a [Actor] / I want [행동] / So that [가치])			
As a 양자 최적화 연구자로서 I want 실제 IBM QPU에서 노이즈로 왜곡된 원시 측정 데이터에 M3 및 ZNE 기법을 적용할 수 있다. So that 하드웨어 잡음 때문에 묻혀 버린 정답 파티 배정 비트열을 복원하고 해의 신뢰성을 확보할 수 있다.			
Priority	☑ Must ☐ Should ☐ Could ☐ Won't	**Estimate (SP)**	5 SP

Acceptance Criteria (Given / When / Then — 핵심 3~5개만 요약)

AC ID	Given (상황)	When (행동)	Then (결과)
AC-01	실제 QPU에서 획득한 원시 측정 카운트 데이터 존재	M3 모듈을 실행	큐비트 판독 오차가 제거된 보정 준확률 분포가 계산된다
AC-02	게이트 에러로 인해 정답률이 낮아진 양자회로 제공	의도적으로 노이즈를 증폭하여 ZNE 선형/다항 외삽을 수행	잡음이 없는 이상적 상태로 추정된 보정 기댓값이 도출된다
AC-03	오류 완화 보정 처리 완료	원시 결과와 보정 결과 비교	이론적 최적해의 검출 빈도 또는 근사비가 원본 대비 20% 이상 향상된다
AC-04	보정 계산 과정에서 물리적으로 불가능한 음수 확률 발생	사후 정규화 알고리즘 적용	모든 비트열 확률의 합이 1.0인 유효한 확률 분포로 변환된다

3.5 E5 – 비교용 일반 컴퓨터 알고리즘 개발

US-05 Brute-force 및 GW기준선 산출 엔진 구현

Epic	E5 비교용 일반 컴퓨터 알고리즘 개발	US ID	US-05
User Story (As a [Actor] / I want [행동] / So that [가치])			
As a 벤치마크 평가자로서 I want 일반 컴퓨터 환경에서 Brute-force와 GW 알고리즘을 실행할 수 있다. So that 양자 알고리즘 연산 결과의 품질을 객관적으로 비교 검증할 수 있는 정답 기준선을 확보할 수 있다.			
Priority	☑ Must ☐ Should ☐ Could ☐ Won't	**Estimate (SP)**	3 SP

Acceptance Criteria (Given / When / Then — 핵심 3~5개만 요약)

AC ID	Given (상황)	When (행동)	Then (결과)
AC-01	파티 갈등 그래프의 인접 행렬 데이터 제공	Brute-force 전수 탐색 모듈 실행	2^N 개의 모든 경우의 수를 검사하여 100% 이론적 최대 갈등 회피 절단값과 정답 비트열을 3초 이내에 도출한다
AC-02	동일한 갈등 그래프 데이터 제공	CVXPY 라이브러리 기반 GW 알고리즘 실행	SDP와 무작위 초평면 기법을 통해 0.878 이상의 근사 성능을 보장하는 고전 분할 결과가 반환된다
AC-03	고전 알고리즘 연산 완료	기준선 데이터 반환 함수 호출	Brute-force 최적값, GW 분할값, 각 알고리즘의 실행 시간이 포함된 JSON 객체가 생성된다
AC-04	고전 알고리즘 출력값 생성	수식 검증 모듈 실행	갈등 회피 점수 계산 공식이 양자 QUBO 목적함수의 평가 방식과 정확히 동일한 수치 체계를 공유함을 확인한다

3.6 E6 – 갈등 관계 드로잉

US-06 인터랙티브 갈등 그래프 생성 및 가중치 편집 웹 캔버스 구현

Epic	E6 갈등 관계 드로잉	US ID	US-06
User Story (As a [Actor] / I want [행동] / So that [가치])			
As a 시뮬레이션 사용자로서 I want 웹 브라우저 캔버스 상에서 마우스 조작으로 참석자 노드를 추가하고 상호간의 갈등 가중치를 에지로 연결할 수 있다. So that 직관적으로 원하는 갈등 회피 네트워크를 구성하고 사전 정의된 시나리오를 불러올 수 있다.			
Priority	☑ Must ☐ Should ☐ Could ☐ Won't	**Estimate (SP)**	3 SP

Acceptance Criteria (Given / When / Then — 핵심 3~5개만 요약)

AC ID	Given (상황)	When (행동)	Then (결과)
AC-01	빈 캔버스 화면 활성화	마우스로 캔버스 더블 클릭	새로운 참석자 노드가 해당 좌표에 생성된다
AC-02	캔버스에 2개 이상의 노드 배치	노드 A에서 노드 B로 마우스를 드래그하여 연결	두 사람 사이에 갈등 연결선이 생성되고 기본 가중치가 부여된다
AC-03	생성된 갈등 연결선 존재	해당 에지를 클릭 후 슬라이더로 가중치 조절	수치와 비례하여 선의 두께와 갈등 강도 라벨이 실시간으로 변경된다
AC-04	메인 캔버스 툴바 화면	프리셋 불러오기 메뉴 선택	미리 정의된 노드 및 가중치 갈등 관계망이 캔버스에 즉각 자동 렌더링된다

3.7 E7- 파티 2분할 시각화 화면

US-07 양자 연산 결과 기반 파티 공간 분할 및 회피 갈등 렌더링

Epic	E7 파티 2분할 시각화 화면	US ID	US-07
User Story (As a [Actor] / I want [행동] / So that [가치])			
As a 시뮬레이션 사용자로서 I want 양자 알고리즘이 산출한 최적 비트열에 따라 참석자 노드들이 두 개의 파티방으로 물리적으로 나뉘는 시각적 애니메이션을 볼 수 있다. So that 어떤 참석자가 어느 파티로 배정되었고 갈등이 어떻게 성공적으로 회피되었는지 직관적으로 파악할 수 있다.			
Priority	☑ Must ☐ Should ☐ Could ☐ Won't	**Estimate (SP)**	3 SP

Acceptance Criteria (Given / When / Then — 핵심 3~5개만 요약)

AC ID	Given (상황)	When (행동)	Then (결과)
AC-01	캔버스에 갈등 그래프가 그려진 상태에서 파티 배정 실행 버튼 클릭	백엔드로부터 최적 비트열 결과가 도착	화면이 좌측과 우측 영역으로 전환되며 노드들이 이동하여 2분할 배치된다
AC-02	노드들의 2분할 배치 완료	회피된 갈등 에지를 렌더링	성공적으로 잘린 에지가 밝은 점선 효과로 강조 표시된다
AC-03	갈등 회피 결과가 화면에 표출	상단 결과 정보 패널을 확인	총 회피된 갈등 가중치의 합과 전체 갈등 중 회피 성공률이 1.5초 이내에 갱신되어 출력된다
AC-04	파티 분할 화면	원본 그래프 보기 토글 버튼 클릭	참석자들의 최초 배치 위치와 파티 2분할 배치 위치 간 전환이 매끄러운 트랜지션 애니메이션으로 동작한다

3.8 E8 – 오류 복원 비교 및 성능 분석 대시보드

US-08 오류 완화 전/후 실시간 복원 시연 및 4자 정량 벤치마크 리포트 표출

Epic	E8 오류 복원 비교 및 성능 분석 대시보드	US ID	US-08

User Story (As a [Actor] / I want [행동] / So that [가치])			
As a 양자 성능 평가자 및 심사위원으로서 I want 웹 대시보드에서 스위치 토글로 오류 완화 전/후의 파티 분할 결과를 비교하고 고전 알고리즘 대비 4자 벤치마크 차트를 확인할 수 있다. So that 하드웨어의 노이즈 영향과 오류 완화 기술의 정량적 복원 효과를 직관적으로 검증할 수 있다.			
Priority	☑ Must ☐ Should ☐ Could ☐ Won't	**Estimate (SP)**	5 SP

Acceptance Criteria (Given / When / Then — 핵심 3~5개만 요약)

AC ID	Given (상황)	When (행동)	Then (결과)
AC-01	하드웨어 결과 화면에서 오류 완화 스위치 OFF	파티 2분할 렌더링 화면 확인	칩 노이즈로 인해 앙숙 관계인 인물들이 같은 파티에 묶이는 갈등 충돌 상태가 붉은색 경고 에지로 강조된다
AC-02	갈등 충돌이 발생한 화면	오류 완화 스위치 ON	정답 비트열 확률 피크가 회복되면서 앙숙들이 서로 다른 방으로 올바르게 재배치되는 애니메이션이 동작한다
AC-03	모든 파이프라인 연산 데이터 수집	상단 벤치마크 리포트 탭을 클릭	Brute-force, GW, QPU 원시값, QPU 보정값 4개 환경의 근사비 비교 그래프가 렌더링된다
AC-04	4자 벤치마크 차트 화면	측정 샷 확률 히스토그램에서 정답 비트열 막대에 마우스 호버	원시 값 대비 보정값의 검출 빈도 향상률과 구체적인 수치 툴팁이 표출된다.

4. 전체 Backlog 목록 (우선순위 정렬)

순위	US ID	User Story 요약	Epic	Priority	SP	배치 Sprint
1	US-01	갈등 그래프 QUBO 수식 변환	E1	Must	3	1
2	US-02	QAOA 최적화 루프 구축	E2	Must	5	1
3	US-05	GW/Brute-force 산출	E5	Must	3	1
4	US-03	실제 IBM QPU 백엔드 연동 및 측정	E3	Must	5	2
5	US-04	M3 및 ZNE 오류 완화 보정	E4	Must	5	2
6	US-06	갈등 그래프 웹 캔버스 빌더	E6	Must	3	2
7	US-07	파티 2분할 공간 시각화	E7	Must	3	2
8	US-08	오류 복원 및 4자 벤치마크 대시보드	E8	Must	5	2
9	US-09	대표 갈등 시나리오 프리셋 제공	E6	Should	2	3
10	US-10	QPU 비동기 실행 상태 실시간 알림	E3	Should	2	3
11	US-11	4자 벤치마크 결과 CSV/JSON 추출	E8	Could	2	4
12	US-12	노이즈 시뮬레이션 파라미터 미세조정 UI	E4	Could	3	4

4.1 Won't — 이번 학기에 포함하지 않는 항

번호	항목	이번 학기 제외 사유	향후 계획

번호	항목	이번 학기 제외 사유	향후 계획
1	실시간 트래픽 가로채기 연동 및 동적 라우팅 엔진	커널 레벨 네트워크 패킷 제어는 물리적 인프라 개발 범위로 본 프로젝트의 핵심 목표에서 벗어남	이번 학기 프로젝트 범위에서 영구 제외
2	대규모 노드 대상 양자 연산 지원	현재 공용 IBM Quantum 백엔드의 큐비트 수 제약 및 큐 대기 시간 지연으로 인해 실시간 렌더링 불가함	시뮬레이션 환경 기반 노드 분할 분산 연산 확장 검토

5. Release Plan

5.1 팀 Capacity 가정

Sprint 길이	2주 (수업 일정 기준)
팀 인원 / 주당 가용 시간	3명 / 주당 30시간(1인당 주 10시간 기준)
Sprint 1 계획 SP	11SP(US-01, US-02, US-05)
Sprint 1 실제 완료 SP (velocity)	← Sprint 1 종료 후 기입
보정된 Sprint 2 계획 SP	13 SP (Sprint1 종료 후 실제 속도에 따라 재조정)

5.2 Sprint 배치

Sprint	주차	Sprint Goal	포함 User Story (ID)	SP 합계	제품 상태
Sprint 0	7주	개발 환경 구축 · 시스템 구조 초안 (기능 개발 없음)	-	0	개발 환경 및 기본 프로젝트 디렉터리 셋업 완료 상태

Sprint	주차	Sprint Goal	포함 User Story (ID)	SP 합계	제품 상태
Sprint 1	8 ~ 11주	갈등 회피 Max-Cut 수학 모델링 및 무잡음 QAOA-고전 기준선 알고리즘 파이프라인 검증	US-01, US-02, US-05	11	알고리즘 CLI 프로토타입
Sprint 2	12 ~ 15주	실제 IBM QPU 백엔드 연동, 노이즈 오류 완화 실증 및 인터랙티브 웹 대시보드 E2E 통합	US-03, US-04, US-06, US-07, US-08	21	최종 웹 플랫폼 완성본

5.3 Release Plan 요약

Sprint	SP	비율	제품 상태
Sprint 1	11 SP	34.4%	알고리즘 검증 가능
Sprint 2	21 SP	65.6%	웹 시각화 및 오류 완화 시연 가능
합계	32 SP	100%	인터랙티브 웹 플랫폼 정식 배포 가능

6. Definition of Done (팀 합의)

완료 기준 (Definition of Done)	합의
코드 작성 완료 + 저장소(main 브랜치)에 병합됨	☑
핵심 기능 단위 테스트 통과	☑
해당 User Story의 모든 Acceptance Criteria 통과	☑

완료 기준 (Definition of Done)	합의
시연 환경에서 실행 확인 + README 갱신	☑
최소 1인 이상의 코드 리뷰 완료	☑
양자 실행 결과 데이터 재현성 확인 및 웹 콘솔 오류 0건 검증	☑

7. Backlog Refinement 계획

수행 시점 · 주기	매 Sprint 중 1회 (Day 8 전후)
참여자	팀원 전원
갱신 기준	실제 칩 대기 시간이나 측정 오차가 예상보다 클 때 일정과 범위 조정 Qiskit 백엔드와 프론트엔드 캔버스 간의 데이터 포맷 불일치 발견 시
갱신 내용	US의 SP 재추정 및 세부 태스크 분할 부가 기능의 우선순위 재정렬 Won't 목록 조정
기록 방법	변경 사항은 표지의 Document Revision History에 반드시 기록, 팀 Github에 동기화 반영

8. 제출 전 자가점검 체크리스트

자가점검 항목	확인
[문서] 프로젝트명 · 조 · 조원 · PO · SM · 제출일자 · GitHub 저장소를 모두 기재했는가?	☑
[DEEP-D] 상위 항목은 AC까지 상세하고, 먼 항목은 개략적으로만 적었는가?	☑
[DEEP-E] 모든 항목에 Story Point가 있는가?	☑
[DEEP-E] 매 Sprint 갱신 계획(Refinement)이 있는가?	☑
[DEEP-P] 가치 · 위험 · 의존성 기준으로 정렬하고 MoSCoW로 표기했는가?	☑
[점검] 맨 위 3~5개 항목을 지금 바로 Sprint에 넣을 수 있는가?	☑
Vision이 5~8줄이고, Success Metrics가 측정 가능한 정량 목표 3~5개인가?	☑
Epic을 UI 화면 이름이 아니라 도메인(업무) 기준으로 묶었는가?	☑
[INVEST] 각 Story가 독립적 · 협상 가능 · 가치 있음 · 추정 가능 · 작음 · 테스트 가능한가?	☑
13 SP를 넘는 스토리를 분할했는가? (CRUD 분리 / 정상 · 예외 흐름 분리 / 데이터 유형별 분리)	☑

자가점검 항목	확인
So that이 기능의 반복이 아니라 사용자 이익 · 가치로 쓰였는가?	☑
AC를 "~할 수 있다"가 아니라 Given / When / Then으로 썼는가?	☑
Story Point를 시간으로 환산하지 않고, Planning Poker로 팀 전체가 합의했는가?	☑
Won't 항목을 명시하여 이번 학기 범위를 닫았는가?	☑
Release Plan의 SP 합계와 Sprint별 배치가 서로 일치하는가?	☑
SP 합계를 팀 Capacity와 대조했는가?	☑
Definition of Done을 팀이 합의하여 문서로 남겼는가?	☑
우선순위를 Product Owner가 최종 결정했는가?	☑
파란색 예시 문구를 모두 삭제했는가?	☑
[부록] 작성 가이드 페이지를 삭제했는가?	☑
문서를 GitHub 저장소에 업로드했는가?	☑